

静态数据驱动的 Wafer-Scale WoW 内存服务共享

第一性原理、架构收束与验证方案

研究目标

联合组织接口、物理连接和静态数据，
使已有 DRAM 服务能够被有效共享。

中心论点

物理上需要支持的方向，不等于执行时需要同时提供的服务。静态数据与伙伴关系既决定哪些资源能够被共享，也决定哪些发送硬件可以复用。

技术基线 v1.0 | 2026 年 10 月 7 日

依据当前对话中的项目报告、上传源码与公开一手资料整理。

本报告不新增仿真或 PPA 实验；新增数值仅来自明确假设下的解析换算。

执行摘要

建议保留研究目标，停止不断更换核心机制。当前最合适的收束对象，是一种静态配对、容量匹配、发送硬件复用的局部内存共享架构。Matching 是描述数据依赖的工具，FIFO 与 gearbox 是执行机制，RTL 是实现证据。它们应共同支撑一个系统问题，而不是分别成为论文题目。

从第一性原理看，DRAM 只能提供已有的原生服务。数据位置固定后，每个请求又必须从指定位置取得不可替代的字节。因此，共享收益来自两个条件：所需资源确实有余量；数据路径能够同时兑现这些余量。仅增加物理连接、增加 FIFO 或提高 nominal bandwidth 都不充分。

Wafer-scale 的贡献背景是一个时间尺度差异：制造时，同一 reticle 模板要支持多个潜在方向；部署时，每个实例可以确定一个共享伙伴；执行时，请求强度变化，但伙伴和数据位置保持固定。已有 configurable source 正是利用部署后的方向互斥，保留两条物理方向，却只保留一套 Shared 发送状态。

现有证据分成三层。第一，固定数据服务模型已经解释了 pair、cycle、接口容量与活动分布的关系。第二，同一 H/plus 上，home、k2、k3 各有自己的性能-成本区域，不能宣布更强共享全面胜出。第三，Nangate45 下的原始综合记录给出 source 面积减少 35.6%，计入相同三个 RX 后为 16.3%；但仍有 hold 和电气违例，且不是整个 wafer 的面积或应用结果。[E1, E2, E3]

本报告建议最终证明的一件事

部署时确定的数据与共享关系，能否在保留有用内存共享收益的同时，降低为多个潜在方向配置服务硬件的成本？

最终证据应由三方比较组成：Private、Paired-Duplicated、Paired-Configurable。先证明共享是否值得，再隔离静态复用的硬件价值，最后评价整个系统的应用-成本关系。

当前最重要的三个收尾动作是：冻结一个完整且物理合法的架构定义；获得双方都合理实现后的可比较硬件成本；并行完成一个真实请求驱动的应用闭环。不再把真实 workload 放在所有局部优化之后，也不以引入 BO、Benders 或更多 DSE 作为收束目标。

阅读顺序：第 1-4 节定义问题并推导共享原理；第 5-7 节落实接口、物理位置和执行合同；第 8-10 节整理证据、成本与最终实验；第 11-12 节给出论文组织和完成计划；附录提供公式核对、证据索引与术语。

目录

执行摘要	1
1 研究对象与边界：先确定这一篇工作要回答什么	3
2 第一性原理：服务、数据和物理连接是三个不同对象	4
3 最小共享构造：为什么静态配对能够利用余量	5
4 架构机会：将制造方向与并发服务分开配置	7
5 容量匹配：为什么接口宽度和数据比例必须一起决定	8
6 主架构规范：从数字源到合法目的地	10
7 执行合同：怎样把“看起来够宽”变成真正的服务	11
8 现有证据：哪些已经成立，哪些还属于完成任务	13
9 统一成本与时钟：局部 RTL 怎样接回整个 wafer	15
10 收束实验：三个主方案、两类证据、一个应用闭环	16
11 论文贡献与相关工作：不扩大题目，也不缩成一个 FIFO	19
12 完成计划：三份有限产物，把 work 真正收尾	20
A 解析核对：用于校验模型，不代替应用结果	22
B 证据版本、代码入口与复现规则	23
C 术语速查	25
参考资料与项目文档	26

1 研究对象与边界：先确定这一篇工作要回答什么

1.1 一个研究目标，而不是一组工具名称

本项目研究 wafer-scale Memory-on-Logic：在一片计算晶圆上，通过 wafer-on-wafer hybrid bonding（WoW HB，晶圆间高密度混合键合）连接另一片存储晶圆。本文关心的是**内存服务如何被组织**，不是发明 DRAM 存储单元、键合工艺或通用片上网络。

建议用如下问题作为全篇唯一研究问题：

在重复 reticle 和合法跨晶圆连接的约束下，怎样联合安排静态数据、出口服务与发送硬件，使繁忙 compute 能使用邻近 DRAM 的空闲服务，并使额外硬件成本可接受？

这里的 reticle 是一个曝光场对应的设计模板及其实例区域。一个 wafer 上有多个 reticle 实例；它们可以具有相同硬件，却采用不同的运行配置。Endpoint 是完整数字数据送入 HB 路径之前或接收之后的接口模块，不等于整个 DRAM controller。Port 是接口的一个出口或入口，HB link 是其连接的物理通路。四者不应混用。

1.2 本篇保留的设计空间

主架构采用已有 H/plus 几何作为工作平台，先固定计算资源、存储总容量与原生服务能力。研究静态伙伴、对象驻留比例、Home/Shared 出口组织，以及潜在共享方向之间的发送硬件复用。这里的静态是**部署期间固定**，不是把每个实例的方向综合成永久常量。

主应用建议从**部署后只读的模型权重或不可变数据对象**开始。它允许一次性完成驻留，避免把缓存一致性、运行时迁移和写协议一起引入。完整请求与返回链仍需在系统模型中存在，不能只假设返回数据会自动出现。

维度	本篇明确研究	本篇不作为新增贡献
物理组织	重复模板、直接 HB 可达性、接入距离与方向成本	新键合工艺、整片晶圆 P&R、通用 NoC
内存组织	有限共享域、静态伙伴、对象内条带与容量匹配	任意远端地址空间、免费迁移或复制
接口实现	完整字、gearbox、背压、静态选向、TX/RX 计费	新 DRAM cell、全面 DRAM 控制器重设计
分析与验证	受约束服务、执行见证、RTL 与应用对照	以某个优化器名称代替架构贡献

表 1 收束边界。已有工具可以继续作为实现手段，但不扩展成新的研究主线。

1.3 为什么仅给 endpoint 换一个大名字还不够

一个共享 FIFO 加方向选择器，单独看确实是小机制。要成为系统架构工作，需要额外回答两个问题：**所限制的共享关系是否仍然保留有用的应用收益；为此增加的完整存储路径成本是否值得**。这不是要求增加更多电路，而是要求把局部节省放回真实使用场景。

本项目已具备这条链的中间两段：静态数据服务分析与可配置发送器的实现。尚需补齐的主要是应用收益，以及从源端到接收端的物理/成本边界。后续章节据此安排工作，而不继续增加机制。

2 第一性原理：服务、数据和物理连接是三个不同对象

2.1 DRAM 带宽来自合法的原生服务，而不是出口数量

DRAM 的一次读包含阵列与外围数据路径的活动。能够连接多个出口，并不意味着原生数字字可以以任意速率产生。SeDRAM 的公开设计把每个独立 channel 的 128-bit RWDL 作为已完成数据合并的数字接口，并通过多个独立 channel 获得并行性。它支持选择一个明确的数字导出边界，但并不证明本项目的任意 bank pooling 或源端 FIFO 已经在 DRAM 工艺中实现。[P1]

同样，bank 内部由多个 subarray 构成，并不意味着各子阵列的全部服务可独立累加。SALP 的分析强调，子阵列具有局部独立性，同时仍共享若干全局外围结构。对本项目，关键是为每个原生服务单元明确数据何时可用，以及公共路径在哪里。[P2]

本文用 μ_b 表示 bank/channel 服务单元 b 的基准服务率。硬件实现所测的有效速率还依赖地址序列、原生供给、排队和发送规则。不能把 μ_b 、出口名义位宽以及测试中实际交付速率当成同一个量。

2.2 固定的是地址位置，不一定是每阶段的访问比例

令 $\ell(c, a)$ 表示 compute c 所需地址 a 的固定驻留位置。对于场景 w ，令 $d_{c,a,w}$ 是读取该地址的字节需求，则实际访问比例应为：

$$A_{cmw} = \frac{\sum_{a: \ell(c,a)=m} d_{c,a,w}}{\sum_a d_{c,a,w}}. \quad (1)$$

ℓ 不随场景变化，但 A_{cmw} 可以随访问热点变化。只有每个对象采用相同比例的细粒度交错、且请求确实按这些比例读取时，才可以简化为所有场景共用 A_{cm} 。

在这个简化条件下，若整个流以 r_c 服务，memory m 必须提供：

$$f_{cm} = A_{cm} r_c, \quad A_{cm} \geq 0, \quad \sum_m A_{cm} = 1. \quad (2)$$

这就是已有理论最值得保留的约束。M0 有空闲带宽，不能代替 M1 发送只驻留在 M1 的字节。原报告也说明，统一对象内 striping 后，仅改变对象权重未必改变 bank 热点；因此不能把不同活动种子的结果差解释成热点缓解。[E2, E4]

存储容量应另外约束。若对象 o 大小为 S_o 、其中 a_{om} 比例驻留于 m ，则必须满足 $\sum_o a_{om} S_o \leq C_m$ 。服务比例并不能自动证明容量可行，更不能把多个 compute 的可达容量求和当作系统总容量。

2.3 服务上限必须保留共享资源

在固定路径的简化模型中，memory、链路和 controller 的约束可写为：

$$\sum_c A_{cm} r_c \leq \mu_m, \quad \forall m, \quad (3)$$

$$A_{cm} r_c \leq b_{cm}, \quad \forall (c, m), \quad (4)$$

$$h_c \leq r_c \leq \min(d_c, \kappa_c). \quad (5)$$

其中 b_{cm} 是路径容量， h_c 是该实验要求的服务下界， d_c 是需求， κ_c 是接收/控制端能力。

若多条路径共用同一 port、串行通路或缓冲，必须再添加共同约束，不能分别赋予独立峰值后直接相加。用统一记号表示：

$$L(g, A) r \leq q, \quad r = h + x \implies L(g, A) x \leq q - L(g, A) h. \quad (6)$$

g 表示几何与通路组织。右侧是**每个具体资源**在基本请求之后的余量，不是一个能随意使用的“全系统闲置带宽池”。

本节结论 真正要提高了，是某个请求所必需的整组资源能够共同提供的服务。Degree、可用边数、FIFO 数和总空闲带宽都只是间接指标。

2.4 几何只决定潜在通路，不能代替端到端可达性

当前项目沿用上游模型：同一 wafer 的 reticle 不直接跨场连线，直接边来自对侧重叠区域。上游还要求每片 wafer 的 reticle 模板相同，并以 H/plus 等布局改变跨层邻接关系。本文使用该平台模型，不把它表述成一切 WoW 制造方式的普遍定律。[P3]

一条可用内存路径还必须同时满足：地址在源 memory 内存在；bank 能接入对应出口；出口与目标 compute 有合法 HB 对齐；目标有接收和重组能力。数据进入 C0 后不能偷偷通过不存在的同层连接送到 C1。图上的 $C - M - C$ 也不自动等于 memory 可以转发数据。

3 最小共享构造：为什么静态配对能够利用余量

3.1 从两个 compute、两颗 memory 开始

考虑两颗单位服务能力的 memory，以及两个 compute。令每个 compute 的对象有 λ 驻留在 home， $1 - \lambda$ 驻留在 peer，采用对称布局：

$$A = \begin{bmatrix} \lambda & 1 - \lambda \\ 1 - \lambda & \lambda \end{bmatrix}. \quad (7)$$

两者同时以速率 1 工作时，每颗 memory 的总负载都是 $\lambda + (1 - \lambda) = 1$ 。因此，在接口和执行规则也可满足的条件下，满载可以平衡。

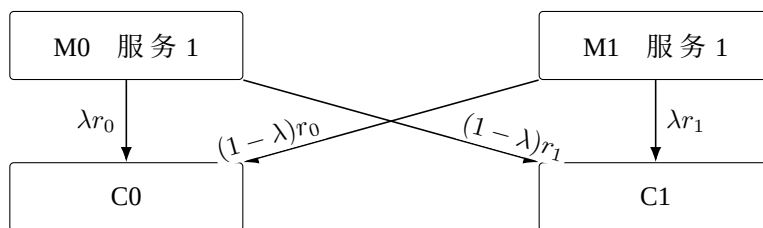


图1 静态配对示意。线表示已存在的直接服务路径；不是运行时复制数据，也不是 C-C 转发。

如果只有 C0 活动，home 和 peer 的路径能力分别为 q_H 、 q_S ，则：

$$r_0 \leq \min \left(\frac{q_H}{\lambda}, \frac{q_S}{1-\lambda}, \kappa_0 \right). \quad (8)$$

在两条路径可同时组合、比例可实现且 controller 不限制的条件下，平衡两项得到：

$$\lambda^* = \frac{q_H}{q_H + q_S}, \quad \gamma^* = q_H + q_S. \quad (9)$$

这里 q_H, q_S 应是可兑现服务，不是名义位宽。单用户的公式还不能代替双用户满载执行检查。

3.2 Shared 是必需数据路径，不是可有可无的加速旁路

取 $q_H = 1, q_S = 0.5$ ，则 $\lambda = 2/3$ ，独占速率为 1.5。两个 compute 同时活动时仍可各得 1；但每人都必须从 peer 取回三分之一的数据，才能完成对象。

因此，**Home 和 Shared 共同承担基础请求；伙伴空闲后释放出来的额外额度才是机会性的**。当前架构并不提供“Shared 断开后仍从 Home 读齐原对象”的功能。要实现这种功能需要复制或不同数据布局，不能在现有论证里默认存在。

同理， $h_c = 1$ 是某一资源/活动模型中要求并验证的服务下界，不等于只靠 Home 硬件就保证任何真实请求达到 1 TB/s。原生供给降低、行冲突或有限对象尾部都会改变这个结论的适用范围。

3.3 Pair 的作用是让竞争伙伴重合

在 50/50、每颗 memory 容量为 1、活动 compute 保底 1 的模型中，一个 memory 同时承担两个活动流时满足 $(r_i + r_j)/2 \leq 1$ ，所以两者只能各为 1。

Pair 让一个 compute 的两个必经 memory 与同一个伙伴竞争。至少含三个 compute 的交替环，则让两条路径与两个不同伙伴竞争。独立活动时，前者更容易同时获得空闲资源。但相关需求可能反过来偏好某些小环；环长超过 pair 后也不一定继续改变平均值。已有构造验证正是对这些受限条件进行核对。[E4, E5]

收束建议：将配对作为主架构的默认共享域，将小环与多 matching 保留为分析和对照，不继续把它们作为新的主方法。真正需要证明的是静态小共享域是否足以支撑目标应用，而不是追求最多 matching。

3.4 把解析均值与应用加速分开

对 $N = 36$ 、恰好 $a = 9$ 个活动 compute、全部已配对的均匀活动模型，已知某 compute 活动时，它的伙伴空闲概率是 $27/35$ 。因此：

$$\mathbb{E}[r_i \mid i \text{ 活动}] = 1 + (\gamma - 1) \frac{27}{35}.$$

(10)

例如 $\gamma = 1.625$ 对应 1.482143，而不是每个场景都得到这个速率。若阶段必须等待最慢的 compute，平均带宽不能直接换算为平均完成时间。附录 A 给出相同模型下的精确区别。

4 架构机会：将制造方向与并发服务分开配置

4.1 三个时间尺度必须分别定义

本项目最有价值的架构切口，不是“可配置比复制更好”这个泛泛判断，而是制造、部署与执行的需求不同。

时间尺度	固定或选择的对象	对硬件的含义
制造	重复模板、端口位置、线宽、潜在物理方向	所有可能在不同实例中使用的方向都要有实体通路
部署	合法伙伴、静态方向寄存器、对象驻留比例	选定本实例需要的服务组合；可识别互斥方向
执行	请求、字返回、仲裁、队列与背压	在固定映射下提供服务，不免费改数据或伙伴

表 2 硬件相同不意味着配置相同；配置固定也不意味着流量固定。

假设模板要支持两个 shared 方向，但每个实例在部署后只使用其中一个。则两条接入线、两组 HB 和对应接收能力仍然需要考虑；FIFO、gearbox 状态与发送控制可以只配置一套，并静态接到所选方向。这是当前 configurable source 的作用。[E1, E6]

4.2 可以复用什么，不能直接省掉什么

资源	当前静态选向能否复用	正确计费方式
Shared TX FIFO / gearbox 状态	可以；前提是同一实例只使用一个 shared 目的地	按一套实现及方向控制计费
两个方向的 HB / 接入线	不能因本实例闲置而从重复模板中删除	按实际制造通路计费
RX 重组与背压存储	不能跨不同目的地默认合并	按系统接收接口的实际实例数计费
未选方向的动态切换	可门控，但不等于已测得功耗下降	同活动文件、同门控假设比较
配置与元数据	仍要存在	区分静态配置、有效长度、真实请求 ID

表 3 复用发送状态，不等于删除物理能力。

4.3 静态复用成立的条件

首先，方向选择必须在 FIFO 和输出拍都处于安全状态时生效。第一版可只允许 reset 或部署初始化时写配置；运行中保持不变。其次，所选方向必须能访问同一合法地址集合，并有足够峰值和背压处理能力。最后，duplicated 与 configurable 的比较必须限制在相同的静态伙伴流量合同下。

Duplicated 在更一般的流量下，可以同时为两个 shared 目的地保留独立状态。Configurable 不具备这个一般功能。本文利用的是**目标部署不需要这部分并发自由度**，不是声称任意输入下两种架构完全等价。

更一般地，制造方向数 D 与最大同时需要的服务上下文数 K 不同，可能产生复用机会。但 $D > K$ 只说明机会存在；实际仍受总发送速率、背压隔离、协议次序和复用切换限制。本文先完成 $K = 1$ 的静态 shared 情形，不急于扩展到动态多上下文。

本节结论 把“数据布局暴露的方向互斥”变成硬件设计依据。论文主张的是目标部署所需的服务可以更便宜地实现，不是可配置接口天然优于一切动态接口。

5 容量匹配：为什么接口宽度和数据比例必须一起决定

5.1 先匹配可兑现服务，再考虑名义宽度

式 (9) 给出的比例是 $q_H / (q_H + q_S)$ ，不是无条件的位宽比。如果原生源每槽能够产生一个完整字，而且队列与 gearbox 能持续工作，出口宽度可能近似决定 q ；若原生供给或完整字接收机会受限，这个近似会失效。

当前代码已经给出直接证据：always-ready 条件下，256/128/128 和 256/160/160 的非对称配置优于等宽 192-bit 候选的某些成本维度；原生供给减半后，继续冻结原来的 2:1 或 8:5 数据比例，则可能输给 50:50 基线。[E9, E10] 这不是架构思想被推翻，而是说明驻留应匹配运行条件下的实际服务，而不是只匹配出厂线宽。

5.2 重复模板产生不同的制造重数

令 home 与每个 shared 方向的宽度相对于原生服务峰值归一化为 H, S 。模板制造一份 Home、两份 Shared 物理出口，所以 lane 预算为 $H + 2S \leq B$ 。设原生长期供给上限为 ρ ，则：

$$q_H \leq \min(\rho, H), \quad q_S \leq \min(\rho, S). \quad (11)$$

由此得到独占服务上界：

$$\gamma \leq \min \left\{ 2\rho, B, \frac{B + \rho}{2} \right\}. \quad (12)$$

证明很直接：两颗原生资源最多合计 2ρ ；有效服务不可能超过全部 lane；又因 $q_H + 2q_S \leq B$ 且 $q_H \leq \rho$ ，两式合并得到 $2(q_H + q_S) \leq B + \rho$ 。这个界不承诺每种缓冲和发起规则都能达到。[E10]

在 $\rho = 1, B = 2.25$ 下，界为 1.625。256/160/160 对应 $H = 1, S = 0.625$ ，配合 $\lambda = 8/13$ 和已验证的完整字序列，在该受限模型中达到此值。[E9]

因此，宽 Home 不是因为 Shared 不重要，而是因为同等峰值的 Shared 物理能力需要制造两份。这个判断针对 lane 成本成立；若缓冲、控制或长线占主导，最优方向可能不同。

5.3 成本向量比一个未校准的总分更有用

已有架构竞争报告还找到了 160/256/256、 $\lambda = 5/13$ 的候选。它与 256/160/160 具有相同的服务均值，但使用更多 lanes 和接入线、较少 endpoint 存储。[E3] 因此，不能把“Home 一定要最宽”当作普遍架构定理。

近期应保留如下成本向量：

$$\mathbf{C} = (A_{TX}, A_{RX}, N_{lane}, B_{buf}, W_{bit-mm}, B_{pipe}, C_{ctrl}). \quad (13)$$

没有可靠转换系数时，不强行将 bit、mm 和 cell area 加成一个“总面积”。同预算比较与同服务目标比较可以并行呈现；最终经过物理实现校准后，再讨论面积或能耗的统一指标。

5.4 有限对象必须真正实现驻留比例

8/13 意味着有限对象或条带组内，应由 8 份 home 字和 5 份 peer 字组成，不意味着每个任意短请求都天然满足这个比例。对象小、访问局部片段或尾部不足一个组时，需要明确定义整数分配、有效长度和偏离。

建议将配置产物写成真实地址映射函数，使用完整字或 cache-line 粒度计算位置。记录每个对象的 home/peer 字数、容量占用和尾部处理。随后由实际请求生成式 (1)，而不是先规定理想比例，再生成总能满足该比例的应用请求。

本节结论 接口与数据比例的联合设计是一项实际机制，但必须锚定原生供给、整数地址映射和完整字执行。不要把位宽最优、平均供给最优和真实应用最优混为一谈。

6 主架构规范：从数字源到合法目的地

6.1 架构图应固定在第一次 HB 之前选向

建议主版本保留一个 Home TX，以及一个可配置 Shared TX。两个物理 shared 方向继续存在，部署时只选其中一个。数据必须在第一次跨 HB 前决定方向；接收端只恢复完整字，不承担隐含转发。

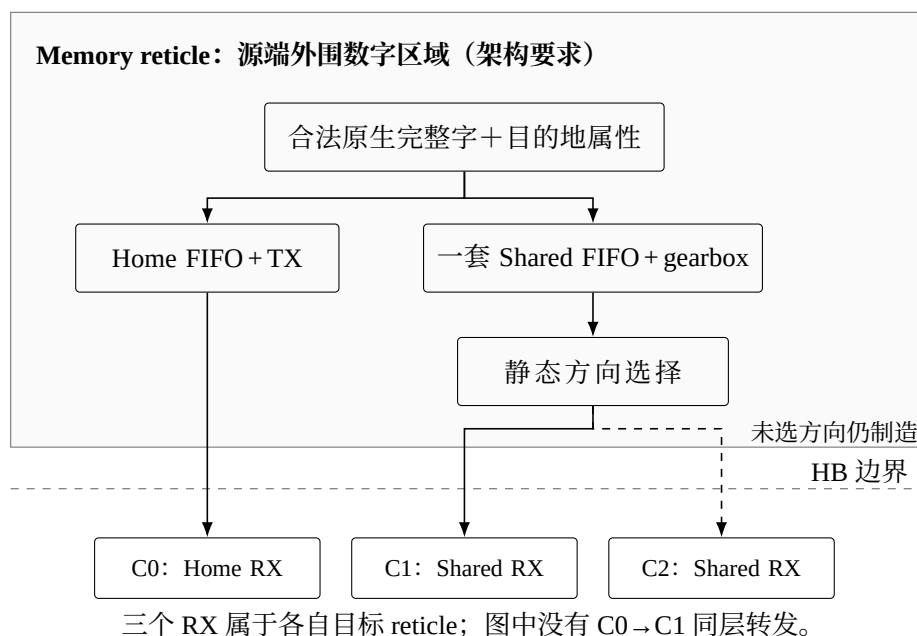


图2 建议冻结的主数据路径。源端逻辑的工艺可实现性需单独落实，不能由此图或 Nangate45 综合替代。

6.2 源端的物理归属需要一项明确决定

当前 RTL 正确性依赖的，是数据在 memory 一侧完成方向选择。将该模块移到 compute C0 后，还需要另一条合法路线才能把数据送往 C1；这会改变原来的两晶圆直接共享架构。

因此，技术基线应明确要求：**源 memory reticle 的外围区域具有所需数字选择、缓冲和驱动能力**。当前 Nangate45 是同库的相对实现证据，不证明这些标准单元能直接放入某个 DRAM 工艺。下一步应通过可用工艺资料、外围实现约束或明确的成本范围落实这一项。不要为逃避这个问题，悄悄添加第三层 logic wafer 或非法同层互连。[E1, E6]

这是一项架构位置与工艺映射决策，不要求现在设计新的 DRAM cell。对于论文，图中位置、参数表中的工艺、以及成本模型必须描述同一条物理路径。

6.3 请求路径、原生供给和返回路径都要存在

当前小 RTL 覆盖 read-return, 从已经产生的完整字开始。系统模型还需要补足读请求的合法到达: compute 发出地址与请求身份, 经控制通路到达有唯一行状态的 bank/controller, 完成合法操作后才产生返回字。

建议 read-only 主版本在系统模型中明确每个 bank 一个命令仲裁/时序实体、合法的请求入口、唯一地址状态和返回目的地。控制带宽可以单独建模，不必第一轮就实现完整 controller RTL，但不

能因为返回路径可达就默认两个 compute 能无成本地同时控制同一 bank。

同样，已发出的 DRAM 读通常不能因 endpoint 的 ready 变低而任意撤销。因此应区分两类输入：**发起机会**可因无 credit 而推迟；**已经完成的数据**必须被保存。真实 DRAM 接入时，应在发命令之前预留返回容量，或明确计入足够的完成队列。当前外生 ready 周期只代表模型机会序列，不等于已经校准的 DRAM 完成 trace。[E9, E10]

6.4 建议的输入与输出合同

原生输入至少携带完整 payload、valid/ready、Home/Shared 属性，以及能确定最终目的地的配置或身份信息。RTL 中不要生成 HHS 等业务序列；那应由冻结地址布局和请求流生成器提供。已有 source 正是消费外部 role/direction。[E6]

HB 数据拍需要 payload、有效长度和 valid/ready 或等价 credit 协议。接收端恢复完整字并保留所属流的顺序。测试中把 tag 编入 payload 的 scoreboard，不自动等于实现了真实 memory request ID、跨源乱序合并或协议报头；这些若为应用所需，必须显式增加并计费。

配置切换只在部署/清空后发生。若保留 per-word shared direction 输入，应检查它始终与静态配置一致；若不需要动态方向，应在 configurable 接口规范中去掉不必要的动态能力，而不是用时序例外掩盖真实动态输入。

7 执行合同：怎样把“看起来够宽”变成真正的服务

7.1 跨字 gearbox 是具体硬件，不是除法

以 256-bit 原生字、192-bit 输出为例，持续满带宽需要某一输出拍包含前一个字的尾部与下一个字的头部。若每个字必须单独发送两拍且不跨字利用剩余位宽，速率只有每拍半个原生字，而不是 192/256。

在早期“槽开始派发完整字，随后发送；D=1 包含正在发送字”的模型中：

$$\frac{R_{D=1}}{\mu} = \frac{1}{\lceil W/w \rceil}. \quad (14)$$

因此 192-bit D1 只有 0.5，而 D2 配合跨字发送可达到 0.75。该式属于这一离散执行规则，不是所有 ready/valid 电路的普遍结论。后续 RTL 有自己的输出拍寄存器和接收 reservoir，必须按实际存储重新计数。[E7, E8, E1]

7.2 平均供给相同，队列需求可以不同

现有原生供给探针对比了每槽一次机会的周期序列。对于 128-bit 出口，两种平均供给都是 0.5 的序列，D1 的实际结果不同：

原生机会周期	D1	D2	D3
10	0.500	0.500	0.500
11110000	0.250	0.375	0.500

表 4 已有完整字探针：完成字/原生槽，不是某款 DRAM 的实测吞吐。来源：[E10]。

如果要求不丢失给定的全部发起机会，可以用累积字计数确定缓冲需要。令 $U_p(t)$ 为到槽 t 应接收的累计完整字， $V_p(t-1)$ 为此前已完成的字，则一种明确槽语义下有：

$$D_p \geq \max_t [U_p(t) - V_p(t-1)]. \quad (15)$$

尾部还剩 1 bit 的字仍占一个完整字位置，不能只把剩余 bit 总量除以 256 来计算占位。工程上应保留这类规则的独立测试，而不是只扫描参数后看吞吐曲线。

7.3 单路能力不能自动拼成一个通用多端口包络

两个独立饱和测试分别测得 q_0, q_1 ，不意味着并发流量能够同时得到 (q_0, q_1) 。公共发起口、FIFO 次序、仲裁与共享接收资源都可能限制它。

对固定的请求顺序、供给和背压，重复状态的周期轨迹可以证明**该条件下一个服务向量能够达到**。它不是对所有地址和停顿序列的通用容量多面体。上层使用这种向量时，应绑定生成条件；若要给 LP 多种可组合服务，必须额外验证并发可组合性，或保留资源耦合约束。[E8, E9]

7.4 从零延迟握手走向长接入线

当前最小 star RTL 是零链路延迟的功能模型。真实 reticle 内接入线可能需要 pipeline，反压也不是瞬时返回。建议下一轮为同一条路径加入明示的延迟与 credit，而不是让远端 ready 组合地跨长线反馈。

一个必要的尺度检查是带宽-反压往返时间积：若发送速率为 R bit/s，反馈往返时间为 T_{rt} ，允许持续满速时需要容纳的在途量通常至少与 RT_{rt} 同量级。完整字占位、突发和协议气泡会增加需要的容量。这个量只给必要的容量尺度，不能替代具体协议证明。

7.5 验证分工

功能验证检查 accepted native words 与 reconstructed RX words 的逐字、逐 bit、目的地一致性；握手验证检查停顿稳定、无溢出/下溢、合法方向以及有限尾部；执行验证检查吞吐与缓冲行为；物理验证检查时序、负载与成本。这四层应相互关联，但不能相互替代。

本节结论 保持已经完成的执行模型和 RTL 路径，不再独立扩 FIFO 课题。只补足真实请求发起、返回容量与长线延迟这些会影响系统结论的连接。

8 现有证据：哪些已经成立，哪些还属于完成任务

8.1 结果必须按模型层次分组

证据层	已有结果	在最终论文中的作用
几何与共享理论	高 degree 不保证资源增加；固定数据决定必经资源；pair/cycle 受活动与容量影响	解释问题，而非新的 DRAM 或 NoC 方案
执行与架构竞争	合同会改变选型；home/k2/k3 各有成本区间；同 lane 下 wire 上限可改变胜者	证明不能只按理想边容量选择设计
RTL 与综合	完整字回放、两种 source 的局部面积对照	证明静态选向复用具有具体实现依据
应用与全路径成本	目前未形成同一版本、同一条件下的完整结果	本篇最重要的收尾证据

表 5 这些阶段是同一研究链的不同层，不应当相乘或拼接为单个 speedup。来源：[E1, E2, E3, E4, E8]。

8.2 合同选型：证明避免选错，而不是架构全面胜出

在固定 H/plus、18 对、50/50 驻留和同一预算下，已有报告给出了三种选型依据的结果：

选择依据	选中实现	实际平均服务	满载每 C
理想独立出口容量	192-bit direct	0.885714	0.5
Buffered 流体合同	192-bit D1	1.000000	1.0
完整字执行	192-bit D2	1.385714	1.0

表 6 random9，单位 TB/s/active compute。预算相同，实际缓冲成本不同。来源：[E8]。

这说明可信模型会改变设计选择。它并不等于 D2 在所有成本维度更好，也没有超过更大 lane 预算下 256-bit direct 的 1.771429。该结果适合用作 motivation 与模型必要性的证据，不应成为最终应用 headline。

8.3 架构竞争：k2 必须保留为强对照

已有完整字与统一成本账本的代表结果如下。各结构独立设计合法静态布局，随后冻结；没有让 k2 沿用 k3 的不适用驻留。[E3]

结构	平均服务	Lane bits	Endpoint bits	接入 bit-mm
Home direct	1.000000	8,192	8,192	120,832.0
k2 direct (2/3 方向)	1.327731	16,384	8,192	241,561.6
k3 A: 256/128/128	1.385714	16,384	24,576	297,497.6
k3 B: 256/160/160	1.482143	18,432	40,960	341,664.0
k3 wide direct	1.771429	24,576	8,192	474,163.2

表 7 random9 精确均值；成本为每 memory reticle 的现阶段模板计数。Endpoint bits 不包括后续 RTL 的全部输出拍、RX、metadata 和物理修复。

A 相对 k2 的服务只增加约 4.37%，同时存储和长线更多；B 提升约 11.63%，也使用更高成本。这组结果不能被删掉，否则 configurable 只与一个昂贵 duplicated 基线比较，会掩盖 private/k2 已有的低成本选择。

8.4 ASIC 证据：一整套 Shared TX 被移除

当前上传报告记录的是执行源码 5d5a958 的单 slice 结果，归档来源是对话中的 718c46e 阶段。Nangate45 typical、2 ns、相同 I/O 约束下：[E1]

计费范围	Duplicated	Configurable	减少
Source 单元面积 / μm^2	17,265.528	11,123.854	35.57%
Source FF 数	1,879	1,197	682 FF
Source + 相同三个 RX / μm^2	37,762.690	31,621.016	16.26%

表 8 修复前映射结果；不是 post-route PPA 或整个系统面积。

已有二次复核把 682 个 FF 对应到一套 Shared TX：512-bit FIFO head/tail、160-bit 输出拍，以及 10-bit count/phase/valid 等状态。它支持节省来自架构去重复，而非把配置绑常量后误删物理方向。[E11]

功能证据包含 14 条成对轨迹，每架构恢复 124,834 个完整字；Verilator、Icarus 和映射后零延迟网表的外部计数与周期一致。这是已测试合同内的有力证据，但不替代全输入形式证明、SDF 时序或 HB 电气验证。[E1]

8.5 Hold 与 RX：收尾实现问题，不再扩成新题目

归档中最差 source hold 分别为 -0.02 ns 与 -0.03 ns ，均落在 native input 到 Home FIFO 的路径。增加 Shared 输出 leaf 不能直接修这条路径。用户随后确认 configurable 需要更多 hold buffer；当前归档未包含该轮修复后面积，因此本报告不推断净差值。[E1, E11]

三个 RX 占 configurable 规定局部总面积约 64.82%。Home RX 的输入输出都是完整 256-bit 字，却使用通用 reservoir；可以对两种架构一起特化。Shared RX 则应保留真实重组和有效长度行为。这样做是加强共同实现，不是把 RX 优化冒充方向复用的额外架构节省。[E11]

本节结论 已有硬件证据支持继续保留 configurable。下一项需要报告的不是更高的修复前百分比，而是双方合理实现、同合同修复后的净面积，以及这个净成本放回应用后是否值得。

9 统一成本与时钟：局部 RTL 怎样接回整个 wafer

9.1 先统一单位，再外推性能

角色接口协议明确使用 256-bit 原生字、1.024 ns 原生槽、每 memory 32 个服务单元。因此，always-ready 峰值为：

$$B_{\text{memory}} = 32 \times \frac{256}{8 \times 1.024 \text{ ns}} = 1 \text{ TB/s.} \quad (16)$$

这是机制模型的带宽标定。ASIC 报告的 2 ns 是同库实现比较的时钟约束。若仅在 2 ns 每周期接收一个完整字，并仍复制 32 份，对应峰值为 0.512 TB/s，而不是 1 TB/s。[E1, E12]

这不是要求否定已有归一化实验，而是需要一张**时钟与服务对应表**：原生 DRAM 的字就绪率、endpoint 逻辑时钟、HB beat 时钟、允许的并行数，以及跨时钟处理方式。只有这些一致后，才能把周期比率转换成物理 TB/s。Pre-layout setup 裕量也不能直接宣布已达到更高 Fmax。

9.2 长期字节吞吐和有限请求时间要分别报告

持续流水状态的服务率适合比较容量；有限对象还会付出首次数据延迟、尾部、credit 往返和重组时间。小对象或串行依赖阶段可能主要受这些项影响。

建议以同一个请求 ID 记录：发起、bank 就绪、TX 接收、各 HB beat、RX 完整字、对象完成 and 任务完成。这样能把收益分成“减少原生等待”“减少输出停顿”“增加初始/尾部延迟”，而不是只记录累计输出字数。

9.3 成本按真实位置拆分，避免重复或漏计

$$C_{\text{path}} = C_{\text{source}} + C_{\text{access wire}} + C_{\text{HB}} + C_{\text{RX}} + C_{\text{control}}. \quad (17)$$

其中 source 的单元面积来自指定库/工艺；接入线用实际分段位宽和长度；HB 包含制造端口与必要 sideband；RX 包含重组与背压存储；控制包含真实请求身份、配置与命令通路。

序列化位置必须与成本一致。如果先以 256-bit 宽总线走到 port，再缩成 160-bit，前段长线仍按 256 bit 计费。只有在线路之前序列化，长线路才可能按 160 bit 计费。当前报告已保留这两种敏感性，后续应选择一份与架构图一致的主实现。[E3, E9]

单 slice 的“一个 source 加三个 RX”是一个规定的比较边界，不是可直接乘以 reticle 数的通用系统模块。全系统必须有实例清单：每个 bank/channel 的 source 数、每个实际目的地的 RX 数、聚合端口与共享控制数量。尤其不能把同一个 RX 同时作为多个 source 的公共接收器，却又忽略聚合与仲裁成本。

9.4 绝对节省比跨层拼接百分比更稳健

现有 source 的绝对节省为：

$$\Delta A_{\text{source}} = 17,265.528 - 11,123.854 = 6,141.674 \mu\text{m}^2.$$

(18)

若两种 source 的净修复增量分别为 R_d, R_c ，则修复后：

$$\Delta A'_{\text{source}} = 6,141.674 - (R_c - R_d).$$

(19)

共同 RX 修复会改变局部分母；长线、HB、compute 与 DRAM 阵列又会进一步改变系统分母。因此，16.3% 不能外推成整个 wafer 面积减少 16.3%，也不能与某一轮服务倍率直接相乘。[E1, E11]

对于不同工艺层，分别报告 memory-side 外围面积、logic-side 接收面积和工艺假设，不将同一个 Nangate45 数字不加说明地折算到 DRAM 层。

9.5 硬件收尾应当以可比较结果为终点

先修现有 source 的真实 input-to-register hold 路径，同时处理 RX 电容/转换时间问题；再将相同 Home RX 特化公平地用于两种架构。局部 P&R 用于验证宽数据路径、时钟树、修复 buffer 与真实 pin 距离的影响，不做 whole-wafer P&R。

OpenROAD 文档建议在 CTS 后使用传播时钟进行 setup/hold 修复，先修 setup，再修 hold，并区分电气约束修复。[P4] 具体流程仍应记录工具版本、corner、SDC、负载与布线阶段。Typical corner 的通过不应写成全工况 signoff。

功耗应使用同一应用或 RTL 活动文件。Duplicated 基线可以采用合理的空闲时钟/数据门控；configurable 的面积收益并不自动保证同百分比动态功耗降低。来源不同的 proxy 和测量应继续分列。

10 收束实验：三个主方案、两类证据、一个应用闭环

10.1 三种主方案分别回答不同的问题

方案	数据与共享关系	硬件组织
Private	对象驻留本地，使用合理 bank 交错	Home-only
Paired-Duplicated	固定合法伙伴，容量匹配的静态对象驻留	每个潜在方向独立 Shared TX
Paired-Configurable	与上一行相同伙伴、驻留、位宽和业务轨迹	一套 Shared TX 静态选向

表 9 主比较矩阵。k2 direct、wide direct 等继续作为性能-成本强参考，不从结果中删除。

Private 对 paired 回答共享是否值得。这里允许各自采用适合硬件的静态物理布局，但逻辑对象、容量、计算资源、请求内容与部署条件相同。

Duplicated 对 configurable 回答部署后互斥能够省掉什么。这里固定相同伙伴、数据、输出宽度、合法输入轨迹和 RX；不能靠同时改变 mapping 把性能差异归因于 endpoint。

Private 对 configurable 回答最终架构是否值得采用。这里要计入长线、HB、源端和接收端成本，并观察应用指标，而不仅是 source 面积。

为量化 static-partner 限制舍弃的灵活性，可以增加一个更一般的动态多方向参考。但它必须使用冻结数据、合法地址可达性与真实资源；只有数据重放置/复制也被放松时，才能称为另一类 oracle，并单列代价。

10.2 用一条可执行应用链替代“随机活动代表应用”

推荐第一条应用线选择**只读模型权重或不可变对象、可以离线部署的推理任务**。固定计算映射与请求调度策略，分别生成 Private 和 paired 的合法地址映射，再由实际对象访问驱动 DRAM 与 endpoint。

MoE 可以作为有输入相关访问的候选。Mixtral 的公开结构会为每个 token 选择部分专家，因此提供了访问随输入变化的实例；但它并不意味着 wafer 上恰好 25% compute 活动，更不意味着本项目一定受内存带宽限制。[P5] Dense 推理或较大 batch 可作为需求更均匀的对照，实际选择由可运行模型与测得瓶颈决定。

需要特别处理**计算通信与内存服务的区别**。本方案的局部共享边并不是完整的 wafer 计算网络。若应用需要跨 reticle 分发 token、汇总结果或做 collective，必须明确平台已有的合法路径，并在全部方案中保持一致、计入成本和延迟。当前禁止同层直连、memory 不转发的条件下，不能凭空添加一个免费全局 NoC。

最小闭环可以先覆盖独立推理任务或一个内存受限阶段；但要准确写出指标层级。只有包含跨层计算依赖、任务间通信与调度后，才报告完整模型/请求的端到端加速。

10.3 真实 trace 要带哪些信息

类别	最小字段	目的
逻辑工作	请求/任务 ID、对象 ID、读偏移、大小、依赖	确保三种方案执行同一份工作
物理映射	compute、memory、bank、地址、Home/Shared	从冻结布局生成真实资源需求
原生服务	命令发起、字就绪、行状态/供给模型版本	区分 DRAM 限制与出口浪费
返回与完成	TX/RX 时间、有效长度、尾部、请求完成	验证不可替代字节及端到端延迟
环境与复现	seeds、布局哈希、队列/时钟配置、source commit	防止测试中重配或混用条件

表 10 请求身份是系统语义；不能用合成的 H/S 周期序列替代真实对象访问。

地址变化会影响 DRAM 行局部性。因此不同驻留方案原则上应重新生成物理地址与原生时序，而不是共用一个与地址无关的 always-ready 源。当暂时只能使用外生 native-ready profile 时，报告为供给敏感性或阶段模型，不能冒充校准后的 DRAM 完成过程。

10.4 先观察机会，再证明收益

在应用比较前，记录每个 compute 的实际读需求、原生 memory 的空闲区间以及伙伴共同忙闲关系。机会性服务只有在目标流可使用的必经资源释放时才有意义；没有请求的总 idle bandwidth 不必全部计为可回收资源。

一个可操作的机会诊断是：对同一实际请求和冻结数据，暂时放松出口机制、但保留原生 memory 和物理可达性，得到该场景的受限乐观值。再恢复真实 endpoint、线延迟和接收约束，分解差额来自哪里。这个值是比较上界，不是最终架构，也不要求新增复杂优化算法。

10.5 主要指标由应用目标决定

对于同步阶段，主指标是阶段完成时间或 makespan；对于多请求推理服务，主指标是吞吐和尾延迟。固定字节 LP 的均值、P5 服务、full-load floor 与空闲利用率用于解释机制，不能替代主指标。

除了相对提升，还应报告绝对时间/服务量、三方案总成本和每个工作负载的分布。测试集在设计前划分；伙伴、地址映射、接口配置在测试前冻结。部署后分布偏移可以作为额外实验，不在看到测试结果后重新选择比例。

10.6 六张主图已经足以组成完整证据链

图	内容	回答的问题
1	三时间尺度与主架构	为什么重复物理方向不要求重复服务状态？
2	固定数据的两-compute 示例	为什么仅有空闲带宽不能替代必需数据？
3	真实需求与伙伴空闲分布	实际应用是否存在可利用机会？
4	三方案应用性能-完整成本	最终方案是否位于有意义的成本-性能位置？
5	同服务合同的 TX/RX/修复成本分解	静态复用究竟省掉了什么？
6	原生供给、相关性与路径延迟敏感性	结论在什么条件下保持、在哪些条件下失效？

表 11 建议的最终结果组织。Matching 全目录、FIFO 全扫描与历史 gate 放附录，不占据主故事。

11 论文贡献与相关工作：不扩大题目，也不缩成一个 FIFO

11.1 一个中心贡献，两项直接相连的机制

建议中心贡献表述为：

面向重复 reticle 的 wafer-scale Memory-on-Logic，提出利用静态数据和伙伴关系组织有限内存共享的架构。通过容量匹配的驻留分布提供可兑现服务，并利用部署后的方向互斥复用发送硬件，在保留目标共享能力的同时减少接口成本。

机制一是容量匹配的数据与出口组织。数据比例决定每条必经路径的负载；真实原生与出口能力决定可行比例。重复模板的方向重数决定资源应怎样分配。

机制二是静态关系驱动的服务硬件复用。硬件保留所有必要的合法方向，却不为当前部署不会同时使用的 shared 方向重复配置全部发送状态。

Matching、完整字模型、RTL 与 ASIC 结果是这两个机制的推导和验证，不应单列成“贡献是用了 ILP”“贡献是写了 RTL”。相关性分析可以为伙伴选择提供依据，但算法若只达到既有强匹配基线，也不需要强行宣称新的优化算法胜利。

11.2 现有相关工作的正确位置

工作	可核实的相关点	本报告如何使用
SeDRAM（2023）	独立 channel 与 RWDL 数字边界、WoW DRAM-logic 集成	原生服务与接口位置依据，不当作本源端成本验证
WoW Network Design（2026 预印本）	重复 reticle、overlap、H/plus 等物理图	采用其平台约束，不申报新 H/plus 或高 radix
H ² EAL（ICCAD 2025）	HB 分布式内存，memory-compute co-placement、页内交错与负载均衡	构造合理 mapping 基线，不把字节交错单独当创新
H ² -LLM（ISCA 2025）	HB 异构架构与 dataflow 联合探索	说明 HB 架构/数据组织已有系统研究
SiloBreaker（TCAD 2026）	已确认 architecture-scheduling co-design 论文记录	作为直接近邻继续比对，不说它“只做调度”

表 12 聚焦工作边界，而非简单列出“别人没做什么”。公开依据见 [P1, P3, P6, P7, P8]。

H²EAL 的页内交错和计算-存储共同放置说明，Private 基线也需要合理的数据组织。SiloBreaker 的标题与索引明确包含架构协同设计；本次没有取得其完整正文，因此不能把它描述成固定架构上的纯软件调度，也不能仅凭名称宣布完全无重合。[P6, P8]

本文的积极定位是：**把部署后的服务互斥落实为源端配置，并在 whole-wafer 使用场景与完整成本中验证。**这比强调“首次 mux/首次 sharing”更准确。静态硬件复用本身也不是 wafer 独有；本工作的特殊价值要来自该平台的重复模板、合法方向和真实应用成本，而不是只靠题目中的 wafer-scale。

11.3 建议的 introduction 因果顺序

第一段提出具体机会：已有 DRAM 服务在不均衡请求下不能被所有需要它的 compute 使用。第二段解释为什么 physical reachability 不够：固定字节、原生供给和实际出口共同限制服务。

第三段提出部署时间尺度：同一硬件模板需要支持不同实例的方向，但实例部署后并不需要全部方向并发。第四段给出架构：静态有限共享域、容量匹配数据分布和 Shared TX 复用。最后用应用-成本和 RTL 实现的不同证据分别支撑系统价值与实现价值。

不要按历史探索顺序写“radix 不行、matching 不行、FIFO 不行，所以换方向”。读者需要的是一个完整问题与解法，不是全部试错日志。

12 完成计划：三份有限产物，把 work 真正收尾

12.1 产物一：冻结一个完整架构与服务合同

交付一份主架构定义，明确物理位置、直接请求/返回路线、伙伴粒度、静态地址映射、Home/Shared 位宽与存储，以及源端可以提供的服务条件。主版本先保持当前配对和可配置发送结构，不引入 bank-group engine、动态伙伴迁移或新的网络。

这份合同至少要能回答：一个具体地址在哪里；由哪个 bank/命令实体服务；从哪个合法出口返回；停顿时数据由谁保存；接收端怎样识别与重组；何时允许更新配置。做到这些，后续指标才对应同一个硬件方案。

12.2 产物二：双方合理实现后的硬件成本

在同一 I/O 条件下分别完成必要的 hold/电气修复；将相同 Home RX 特化用于双方；根据实际路径考虑 Shared RX 的固定粒度实现。保存功能、映射后回放、约束、修复前后网表与面积，并在必要的小块 P&R 中验证本地宽线和时钟影响。

这项任务的完成标准是得到可用于系统账本的可信成本与时钟点，不是追求无限降低 gate count。没有输出端瓶颈证据，不优先新增 dual-leaf；没有应用并发需求，不扩展复杂仲裁。

12.3 产物三：三方案在一个真实请求分布上的闭环

硬件收敛与应用实验并行推进。用冻结数据和实际请求地址驱动原生时序/供给、endpoint、链路及接收端，完成 Private、Paired-Duplicated、Paired-Configurable 三方比较，同时保留 k2 与宽 direct 的强成本参考。

先报告一个明确层级：完整请求、多任务服务，或一个标注清楚的模型阶段。随后只补该主结论需要的跨-reticle 通信与计算依赖。若只能证明内存阶段改善，就不要把它写成完整 LLM 加速，但也不需要为追求大标题增加未验证机制。

工作包	完成判据	不继续扩展的内容
架构与合同	每份数据、每条请求/返回路线、每项缓冲归属可明确追踪	新堆叠层、任意全局共享
硬件成本	同合同功能成立；修复成本明示；时钟与带宽对应一致	无限 FIFO/RX 微优化
应用闭环	冻结映射下三方案完成相同工作，给出性能与完整成本	先发明新 scheduler 再挑流量
论文组织	主张、机制、证据逐项对应；历史结果标清模型与版本	将不同阶段倍率拼成一个结果

表 13 不是要求所有场景零退化，而是要求最终取舍明确、可复现且与目标应用有关。

12.4 不同结果应该怎样解释，而不是继续换题

若 configurable 保留目标服务并降低总成本，主架构成立。若源端面积节省被长线/RX 稀释，但仍在某个预算区间有优势，则报告该成本区间，不扩大节省百分比。

若 k2 在低预算更优，应保留它，说明需要怎样的服务目标才值得 k3/configurable。若真实需求长期均匀，private 或窄共享可能就是合理选项；若实际访问中伙伴常同时忙，则调整同一架构的部署关系，而不是马上加入新 endpoint。

若源端工艺或合法请求通路无法支撑当前图示，那是需要解决的架构边界，不能由更多 LP 或 RTL 功能测试掩盖。把这项问题定位准确，比继续扩大无物理约束的设计空间更有效。

12.5 最终收束句

本工作的中心不是设计一个更小的 FIFO，也不是求更多 matching。
它研究的是：数据和共享伙伴在部署时确定后，怎样让物理上广泛可达的 WoW 系统，只配置实际需要的并发服务硬件，同时保留有用的 DRAM 共享收益。
静态数据决定必须提供的服务；容量匹配决定提供多少；可配置发送器决定用多少重复硬件提供。应用-成本对照把三者闭合起来。

A 解析核对：用于校验模型，不代替应用结果

A.1 为什么满载平衡会出现 matching mixture

在相同数量、单位服务的 compute 与 memory 中，若每个 compute 全负载要求 1，且每行 A 和为 1，则总需求等于总原生能力。memory 列负载均不超过 1，且所有列和的总和也为 N ，因此每列和必须为 1。

这时 A 为双随机矩阵，可用受物理图支持的 perfect matching 的非负混合表示。这个表示描述静态数据比例，不是在运行时轮流把整份数据挪到不同 matching。它是当前对称模型的表达方式；异构容量、不等需求、缺陷和不等数量时，需要直接用原资源约束，而不是强行要求 perfect matching。[E4]

A.2 均匀固定活动数下的配对均值

对全部配对的 N 个 compute，恰好 a 个均匀活动。已知 i 活动，剩余 $N - 1$ 个节点中有 $N - a$ 个空闲，因此伙伴空闲概率为 $(N - a)/(N - 1)$ 。保底为 1、独占速率为 γ ，得到式 (10)。

若 $N = 36, a = 9$ ，则：

独占 γ	平均服务	平均公共速率	等工作量阶段时长/基线
1.500	1.385714	1.132210	0.911860
1.600	1.462857	1.158652	0.900842
1.625	1.482143	1.165263	0.898300
2.000	1.771429	1.264421	0.867790

表 14 本报告解析换算。条件是固定对称配对、单位基线、同量工作和上述流体服务规则；不是新仿真，更不是应用加速数据。

无任何一对同时活动的概率为：

$$P_0 = \frac{2^a \binom{N/2}{a}}{\binom{N}{a}}, \quad a \leq N/2. \quad (20)$$

该事件发生时所有活动 compute 可独占，否则至少有一个活动 pair 维持基线。因此平均公共速率为 $1 + (\gamma - 1)P_0$ ；等工作量阶段的平均归一化时长为：

$$\mathbb{E}[T/T_0] = (1 - P_0) + P_0/\gamma. \quad (21)$$

注意，这不等于平均速率的倒数。

A.3 为什么更多必需伙伴未必更好

若一个流必须从两个各有不同竞争者的 memory 取齐数据，在独立活动概率 p 下，同时空闲的概率可能是 $(1 - p)^2$ ；若两份数据共享同一个竞争伙伴，则是 $1 - p$ 。但在相关需求中，应直接使用联合概率 $\Pr(\Gamma_i \cap S = \emptyset \mid i \in S)$ ，不能只代入竞争者数量。

取消保底后, 资源分配目标也会改变。例如连续三个活动节点可能由 (1, 1, 1) 变为 (1.6, 0.4, 1.6), 提高总服务但降低中间节点服务。这就是为什么优化均值、最大化公共完成和最小服务约束必须分别定义。[E4]

A.4 固定路径份额与瞬时数据流的区别

式 (2) 描述平均流或完整工作量的固定字节比例, 不要求每个时钟都同时按比例传输。执行器可以有 burst 和相位差, 但累计接收与交付必须守恒, 且最后完成的对象包含全部必需字节。

若有多条合法输出通向同一个最终 compute, 可以另设路径份额; 若另一个输出只通向不同 compute, 则不能因为它空闲就计为本流可聚合容量。路由自由度与数据复制自由度必须分开。

B 证据版本、代码入口与复现规则

B.1 本报告实际使用的证据

本次检查了当前挂载的 w2w-memory-main.zip 中的报告、方法与理论文件, 并读取了已有 configurable 二次复核材料。未重新运行 RTL、LP、综合、STA 或 P&R; 表格中的实验数值来自归档, 附录 A 的数值为解析重算。

项目更新提交与实验执行提交可以不同。例如 718c46e 是归档阶段, ASIC 报告内的执行源码是 5d5a958; 图文更新不应改写原实验 provenance。后续用户确认的 hold-buffer 差异属于新状态, 但缺少修复后定量产物时, 不能自行补出面积和功耗。

索引	文件与定位	用途/证据等级
E1	docs/reports/ENDPOINT_ASIC_SLICE_REPORT.md	原始 ASIC/功能报告; 执行 5d5a958。
E2	docs/reports/SERVICE_DRIVEN_REPORT.md	bank sharing 与配对、成本和训练过拟合记录; ea8a386。
E3	docs/reports/ARCHITECTURE_COMPETITION_REPORT.md	home/k2/k3 执行-成本竞争; 806fe829。
E4	docs/theory/MATCHING_THEORY_FORMULATION.md	固定字节、matching、pair/cycle 与模型前提。
E5	docs/reports/CYCLE_CONSTRUCTION_REPORT.md	构造、错误拒绝、LP 与小系统覆盖校验。
E6	docs/reports/SHARED_EGRESS_ARCHITECTURE_DECISION.md	方向互斥、读返回边界与源端物理位置。
E7	docs/reports/ENDPOINT_BRIDGE_REPORT.md	Serial/Buffered/FIFO 与 wafer 服务桥接。

索引	文件与定位	用途/证据等级
E8	docs/reports/CONTRACT_SELECTION_REPORT.md	同预算三种模型选型；8b27f7b。
E9	docs/reports/ROLE_INTERFACE_REPORT.md	位宽、比例、供给敏感性；dda731f。
E10	docs/theory/ROLE_INTERFACE_PRINCIPLES.md	成本界、完整字占位与 burst 推导。
E11	CONFIGURABLE_ENDPOINT_EVIDENCE_DRIVEN_REVIEW.md	已有二次源码/STA/网表复核；不是新的物理实现结果。
E12	docs/methods/ROLE_INTERFACE_METHOD.md	1.024 ns 原生槽、D 语义与注册协议。

B.2 后续只维护一个实验事实清单

每条性能或硬件结果应绑定：source commit、输入 trace 哈希、设计对象哈希、静态地址映射、目标/保底、时钟和单位、执行/流体层级、计费范围、未解决项。Report、绘图与论文表格从同一事实清单生成，不在文中手工混合不同版本。

独立校验仍需保留。统一 ResourceLedger 有利于维护，但若 checker 完全复用同一矩阵生成器，就只能检查数值可行性，不能发现遗漏某项资源的模型错误。应保留两-compute 手算例、独立累计字节/队列计数、负测试，以及 trace 级请求恢复作为不同来源的校验。

B.3 需要关闭的历史表述

不再使用的说法	统一后的说法
Shared 只是可选加速路径	固定数据下 Shared 可能是必需路径；额外服务才是机会性的。
Degree 或 matching 越多越好	取决于必经资源的共同余量、容量和数据比例。
完整字 probe 就是通用服务包络	它是给定源/顺序/背压下的执行见证，组合需要额外检查。
Endpoint 放 logic 侧就沿用旧可达性	方向选择位置必须与实际合法物理路径一致。
2 ns 通过就兑现 1 TB/s 模型	必须明确时钟、并行数与原生供给的换算。
16.3% 就是 wafer 面积节省	它是指定局部计费边界内的修复前映射比例。
用了 Benders/BO 就是 SOTA	本篇先证明架构与应用价值；工具名称不是贡献。

表 16 这些修正用于统一整篇论证，不要求重新开启已收束的支线。

C 术语速查

术语	在本文中的含义
WoW	两片晶圆通过键合堆叠；不自动等于整片晶圆是一个计算系统。
Wafer-scale	多个 reticle 实例共同构成所研究的平台与资源组织范围。
Reticle/template	曝光场对应的设计模板；多个实例可共用硬件但有不同部署配置。
HB / hybrid bonding	晶圆间的细粒度物理连接；不是完整内存服务能力。
Bank/channel 服务单元	模型中提供原生完整字的资源；必须注明与真实 DRAM 组织的映射。
Endpoint	把完整数字字送上物理路径或接收重组的局部模块，包含队列、gearbox、握手等。
Gearbox	在不同总线位宽间连续分割/拼接数据的转换结构；可以跨原生字边界。
Home / Shared	同一对象相对其 compute 的本地/伙伴来源角色；不等于必需/可选。
Static layout	部署后冻结的地址位置；不同对象请求仍可造成不同瞬时需求。
Service contract	地址可达性、共享资源、完整字与背压等条件下可提供的服务约束。
Execution witness	给定条件下实际执行得到的服务轨迹或重复周期，不是所有流量的普遍保证。
Matching mixture	用多个一一配对矩阵表示静态字节比例的方法，不代表运行时免费迁移。
Pareto 前沿	不存在另一设计在所有所报成本/性能维度都不差、且至少一项更好的候选集合。
Hold repair	为满足最小延迟约束进行的物理/门级修复；不同于 RTL 数据功能错误。

参考资料与项目文档

公开资料核对日期：2026-10-07。P 为公开一手资料，E 为用户项目证据。研究结论与建议在正文中分开。对未取得完整正文的条目，仅使用已核实的书目信息/公开说明。

- [P1] Song Wang, Xiping Jiang, Fujun Bai, et al. *A True Process-Heterogeneous Stacked Embedded DRAM Structure Based on Wafer-Level Hybrid Bonding*. Electronics, 12(5):1077, 2023. 核对 §2.1 的 RWDL/channel 描述。<https://doi.org/10.3390/electronics12051077>
- [P2] Yoongu Kim, Vivek Seshadri, Donghyuk Lee, Jamie Liu, Onur Mutlu. *Exploiting the DRAM Microarchitecture to Increase Memory-Level Parallelism*. arXiv:1805.01966, 2018; 对 ISCA 2012 SALP 工作的作者回顾。<https://arxiv.org/abs/1805.01966>
- [P3] Patrick Iff, Tommaso Bonato, Maciej Besta, Luca Benini, Torsten Hoefler. *Network Design for Wafer-Scale Systems with Wafer-on-Wafer Hybrid Bonding*. arXiv:2603.05266, 2026. 本报告平台对照使用 v1 的 §2–4; arXiv 已有 v2, 不使用 v1 的占位出版字段作正式会议信息。<https://arxiv.org/html/2603.05266v1>
- [P4] OpenROAD. *Gate Resizer Documentation*. 官方 repair_design、repair_timing 与传播时钟说明。<https://openroad.readthedocs.io/en/latest/main/src/rsz/README.html>
- [P5] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. *Mixtral of Experts*. arXiv:2401.04088, 2024. 仅用于说明输入相关专家选择，不作为本项目应用收益证据。<https://arxiv.org/abs/2401.04088>
- [P6] Zizhuo Fu, Xiaotian Guo, Wenxuan Zeng, et al. *H²EAL: Hybrid-Bonding Architecture with Hybrid Sparse Attention for Efficient Long-Context LLM Inference*. ICCAD 2025; arXiv:2508.16653v2. 核对 §IV-B 的 memory-compute co-placement 与页内交错。<https://arxiv.org/html/2508.16653v2>
- [P7] Cong Li, Yihan Yin, Xintong Wu, et al. *H²-LLM: Hardware-Dataflow Co-Exploration for Heterogeneous Hybrid-Bonding-based Low-Batch LLM Inference*. ISCA 2025, pp.194–210. 本次使用出版元数据及机构摘要。<https://doi.org/10.1145/3695053.3731008>
- [P8] Junyi Wu, Xin Zhao, Jiayi Huang, Fengbin Tu. *SiloBreaker: Breaking the Resource Silo in Hybrid Bonding via Architecture-Scheduling Co-Design*. IEEE TCAD, 2026. 本次确认出版社记录与作者发表目录; 未取得完整正文, 不据此断言某一局部机制不存在。<https://doi.org/10.1109/TCAD.2026.3737060>
- [E1] 项目原始报告: docs/reports/ENDPOINT_ASIC_SLICE_REPORT.md。执行源码 5d5a958; 同库功能、面积和未收敛时序记录。当前上传归档。
- [E2] 项目原始报告: docs/reports/SERVICE_DRIVEN_REPORT.md。执行源码 ea8a386; 固定字节、完整 bank 配对、成本与泛化结果。
- [E3] 项目原始报告: docs/reports/ARCHITECTURE_COMPETITION_REPORT.md。执行源码 806fe829; home/k2/k3 的注册设计族、精确期望与成本前沿。
- [E4] 项目理论: docs/theory/MATCHING_THEORY_FORMULATION.md。固定数据、平衡 matching、竞争关系与服务目标。
- [E5] 项目原始报告: docs/reports/CYCLE_CONSTRUCTION_REPORT.md。局部配置、全局覆盖与服务 LP 的独立核对。
- [E6] 项目架构记录: docs/reports/SHARED_EGRESS_ARCHITECTURE_DECISION.md。静态选向、read-return 验证与物理位置边界。
- [E7] 项目原始报告: docs/reports/ENDPOINT_BRIDGE_REPORT.md。Serial/Buffered 与完整字执行到 wafer 的首次闭环。

- [E8] 项目原始报告: docs/reports/CONTRACT_SELECTION_REPORT.md。执行源码 8b27f7b; 16 个接口候选及不同合同选型。
- [E9] 项目原始报告: docs/reports/ROLE_INTERFACE_REPORT.md。执行源码 dda731f; A/B 非对称接口、固定比例与原生供给敏感性。
- [E10] 项目理论: docs/theory/ROLE_INTERFACE_PRINCIPLES.md。原生供给-重复模板成本界、完整字缓冲及突发阈值。
- [E11] 对话既有复核文档: CONFIGURABLE_ENDPOINT_EVIDENCE_DRIVEN_REVIEW.md。基于 718c46e 上传包回溯 FF、STA 和 RX 状态; 为二次分析, 不视为新一轮综合结果。
- [E12] 项目注册方法: docs/methods/ROLE_INTERFACE_METHOD.md。规定 256-bit/1.024 ns 原生槽、队列 D 语义、静态序列与成本边界。